

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Towards AI · [Follow publication](#)

Understanding the AI Code Is the New Technical Debt

AI Architect's Notes · Week 01 · Honest Takes

6 min read · 4 days ago



Shwetha Kumari A

Follow



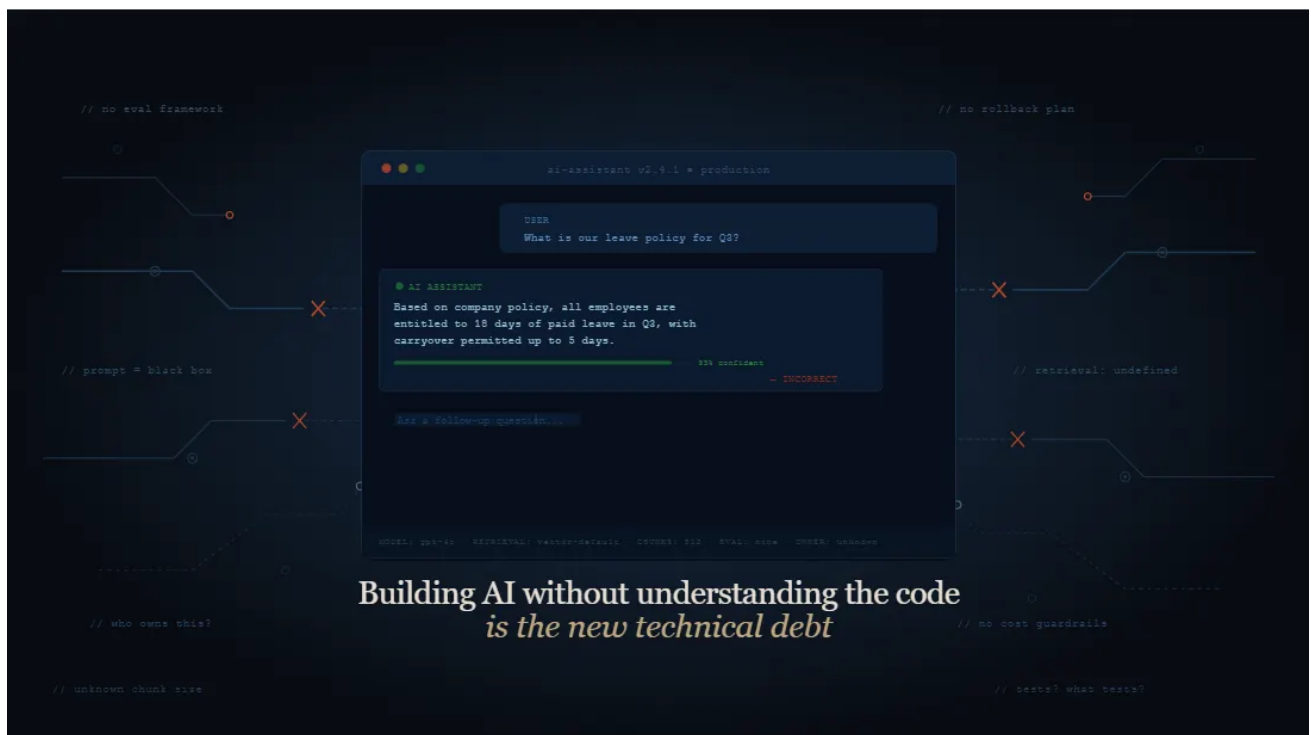
Listen



Share



More



93% confident. Completely wrong. Nobody knows why.

We have confused the ability to assemble AI with the ability to understand it. These are not the same skill. And the gap between them is quietly becoming the most expensive line item in enterprise AI.

A Story That Is Playing Out Everywhere Right Now

Picture a mid-sized enterprise. Smart people, real ambition, genuine pressure to ship AI.

A non-developer discovers that he can connect an AI API to a SharePoint library, write a few prompts, and have a working chatbot in three days. It answers questions about internal policy. Leadership is thrilled. It gets demoed in the all-hands. A VP calls it a “game changer.”

Six weeks later, the chatbot starts giving confidently wrong answers about leave policy — the kind of wrong that creates HR escalations. Nobody can explain why. The analyst who built it has moved on to the next initiative. The engineer who was “oversight” never looked at the retrieval logic. The vendor who sold the middleware says the prompts need tuning.

Three months of trust, gone. The system gets quietly switched off.

Here is what nobody says out loud in the post-mortem: **the problem was not the technology. It was that nobody in the chain actually understood what they had built.**

Speed is seductive. But when your AI system breaks at 2 AM and nobody in the room can explain what it is doing, you are not moving fast. You borrowed time — and the bill just arrived.

This Is Not About Blaming Anyone

The analyst was not wrong to build fast. Speed is a legitimate competitive advantage. The engineer was not lazy — they were under-resourced and over-

allocated. Leadership was not naive — they were responding to real board-level pressure to show AI progress.

The villain here is not a person. It is a **culture that has normalized shipping AI systems nobody fully understands** — because the tools make it easy, the demos look convincing, and the questions that matter are slow, uneasy and often ignored.

We have spent years building engineering cultures that reward shipping. That instinct is not wrong. But AI systems have a property that most software does not — **they fail softly, confidently, and invisibly** until they do not.

A broken API throws an error. A misconfigured database fails loudly. A language model that has drifted, retrieved the wrong context, or been given an ambiguous prompt? It gives you an answer. It sounds certain. It is wrong. And nobody in the room knows why.

The Three Gaps That Are Quietly Accumulating

Gap one — The prompt gap

Most teams treat prompts like form fields. You type something, it works, you ship it. What they do not understand is that a prompt is logic. It has edge cases, failure modes, and interactions with retrieval context that are non-obvious. When a prompt breaks in production it breaks differently every time, depending on what the user typed, what got retrieved, and what the model's current behavior is on that particular query pattern.

If nobody on your team can read a prompt and reason about why it might fail — that is technical debt. Silent, growing, compounding.

Gap two — The retrieval gap

RAG — retrieval-augmented generation — is now the default architecture for enterprise AI. The idea is simple: retrieve relevant documents, pass them to the model, get a grounded answer. The execution is not simple at all.

Chunk size matters. Overlap strategy matters. Whether you use dense vectors, sparse BM25, a hybrid of both or vector less RAG matters. Reranking matters. Metadata filtering matters. Most teams using RAG have made all of these decisions implicitly — by accepting defaults — and have no idea what those defaults are or whether they are appropriate for their data.

That is not a technology problem. It is a comprehension problem.

Gap three — The evaluation gap

How do you know your AI system is getting better or worse after a change? If the answer is “we test it manually and see if it feels right,” you do not have a system. You have a science project.



Evaluation frameworks — even simple ones — require understanding what the system is doing well enough to define what good looks like. That understanding cannot come from someone who only knows how to assemble the parts.

What Constructive Looks Like

This is not an argument for slowing down. It is an argument for building differently.

Make comprehension a team norm, not a personal virtue

Every AI system your team ships should have at least one person who can answer three questions without consulting documentation: What happens when this retrieves the wrong context? What does this prompt do when the user's query is ambiguous? How would we know if this system is quietly degrading?

If nobody can answer those questions, the system is not ready — regardless of how good the demo looks.

Separate “building” from “owning”

The non-developer who builds the prototype is not the same person who should own it in production. This is not a slight on non-developers — it is a structural problem. Production AI ownership requires understanding the failure modes, the cost model, the retrieval logic, and the evaluation strategy. Make that ownership explicit, named, and resourced.

Run a fire drill before you go live

Before any AI system goes to production, run a deliberate failure exercise. Ask the system questions it should not be able to answer. Give it malformed input. Overwhelm it with concurrent users. Watch what happens — not whether it fails, but *how* it fails and whether the team can explain why.

If the failure surprises you, you did not understand the system well enough to ship it.

Invest in one hour of depth per week

The most practical thing a team can do is build a standing habit of going one layer deeper. Not reading vendor docs. Not watching demo videos. Actually understanding: How is the retrieval impacting the responses? What is the token budget in this context window? What does this reranker actually optimize for?

One hour a week, over three months, compounds into a team that actually understands what it is building.

The Real Cost of Not Doing This

Technical debt from misunderstood code is expensive. You pay it in bugs, rewrites, and maintenance cycles.

Technical debt from misunderstood AI is different. You pay it in **trust** — from users who got a wrong answer, from stakeholders who got a failed demo, from a business that invested in a system that nobody can debug or improve.

Trust, once lost in an AI system, is extraordinarily hard to rebuild. One confident wrong answer in front of the wrong person can derail a roadmap by six months.

The organizations that will win with AI over the next three years are not the ones that shipped the most prototypes in 2024. They built fewer systems they couldn't explain, and more that they could.

Closing Thought

The tools have never been more powerful. The abstractions have never been higher. You can ship an AI system today that would have taken a team of ten a year to build in 2020 — and ship it without understanding a single line of what is happening underneath.

That is remarkable. It is also dangerous.

Assembly is not architecture. Speed is not understanding. A working demo is not a system you own.

The next time your team ships an AI feature, ask one question before it goes live: if this breaks at 2 AM, who in this organization can explain why — and fix it?

If the answer is nobody, you have not shipped a solution. You have scheduled a crisis.

What is the most expensive “we didn’t understand what we built” moment you have seen in AI? I’d genuinely like to know — drop it in the comments.